

API Documentation: Housing Market & Rental Insights API

1. Overview

This API provides: - CRUD endpoints for **regions** and **listings** - Analytics endpoints for rent and affordability insights - Scenario simulation, price-trend analysis, and composite risk scoring

2. Base URL and Local Run

- Base URL: `http://127.0.0.1:8000`
- Swagger: `http://127.0.0.1:8000/docs`
- ReDoc: `http://127.0.0.1:8000/redoc`

Run locally:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

3. Authentication

Write endpoints require bearer token auth via **Authorization** header.

Example:

```
curl -X POST http://127.0.0.1:8000/auth/token \
-H "Content-Type: application/x-www-form-urlencoded" \
-d "username=admin&password=adminpass"
```

Then call write endpoints using:

```
curl -X POST http://127.0.0.1:8000/regions/ \
-H "Authorization: Bearer <ACCESS_TOKEN>" \
-H "Content-Type: application/json" \
-d '{"name":"Leeds","ons_code":"E08000035","average_income":29500}'
```

Read endpoints are public, while write endpoints require **write** scope. Demo credentials: - `admin/adminpass` -> `read write` - `viewer/viewerpass` -> `read only`

4. Response and Error Conventions

- Success responses are JSON (except 204 No Content delete responses).
- Error shape is standardized:

```
{
  "error": {
```

```

    "code": 404,
    "message": "Region not found"
  }
}

```

- Validation errors:

```

{
  "error": {
    "code": 422,
    "message": "Validation failed",
    "details": [
      {
        "type": "greater_than",
        "loc": ["query", "monthly_rent"],
        "msg": "Input should be greater than 0",
        "input": -1,
        "ctx": {"gt": 0.0}
      }
    ]
  }
}

```

Unauthorized example (401):

```

{
  "error": {
    "code": 401,
    "message": "Missing bearer token"
  }
}

```

Forbidden example (403):

```

{
  "error": {
    "code": 403,
    "message": "Insufficient scope for write operations"
  }
}

```

Conflict example (409):

```

{
  "error": {
    "code": 409,
    "message": "Region with same name or ONS code already exists"
  }
}

```

```

}

Common status codes:
- `200 OK`
- `201 Created`
- `204 No Content`
- `401 Unauthorized`
- `403 Forbidden`
- `404 Not Found`
- `409 Conflict`
- `422 Unprocessable Entity`

```

5. Data Models

```

### Region
```json
{
 "id": 1,
 "name": "Leeds",
 "ons_code": "E08000035",
 "average_income": 29500.0
}

```

### Listing

```

{
 "id": 1,
 "region_id": 1,
 "price": 950.0,
 "bedrooms": 2,
 "listing_type": "rent"
}

```

## 6. Endpoints

### 6.0 Auth

**POST /auth/token** Issue bearer token using form credentials. - Content-Type: application/x-www-form-urlencoded

Form fields: - username - password

Success: 200

```

{
 "access_token": "<token>",
 "token_type": "bearer",
 "expires_in": 3600,

```

```
"scope": "read write",
"role": "admin"
}
```

Errors: 401 invalid credentials

## 6.1 Regions

**POST /regions/ (auth required)** Create region.

Body:

```
{
 "name": "Leeds",
 "ons_code": "E08000035",
 "average_income": 29500
}
```

Success: 201

```
{
 "id": 1,
 "name": "Leeds",
 "ons_code": "E08000035",
 "average_income": 29500.0
}
```

Errors: - 401 missing/invalid bearer token - 403 insufficient scope (non-writer token) - 409 duplicate region name/ONS code - 422 validation failed

**GET /regions/** List regions with pagination/filter/sort.

Query params: - skip (int, default 0) - limit (int, default 100, max 500)  
- name\_contains (string, optional) - sort\_by (id|name|average\_income, default id) - sort\_order (asc|desc, default asc)

Success: 200

```
[
 {
 "id": 1,
 "name": "Leeds",
 "ons_code": "E08000035",
 "average_income": 29500.0
 }
]
```

**GET /regions/{region\_id}** Fetch one region.

Success: 200

Errors: 404 if not found

**PUT /regions/{region\_id} (auth required)** Update full region record.

Body:

```
{
 "name": "Leeds Updated",
 "ons_code": "E08000035",
 "average_income": 30000
}
```

Success: 200

Errors: 401, 403, 404, 409, 422

**DELETE /regions/{region\_id} (auth required)** Delete region.

Success: 204 (empty body)

Errors: - 401 - 403 - 404 - 409 if listings still reference region

## 6.2 Listings

**POST /listings/ (auth required)** Create listing.

Body:

```
{
 "region_id": 1,
 "price": 900,
 "bedrooms": 2,
 "listing_type": "rent"
}
```

Success: 201

Errors: 401, 403, 404 (region missing), 409, 422

**GET /listings/** List listings with pagination/filter/sort.

Query params: - skip (int, default 0) - limit (int, default 100, max 500) - region\_id (int, optional) - listing\_type (rent|sale, optional) - min\_price (float, optional, >0) - max\_price (float, optional, >0) - bedrooms (int, optional, >=0) - sort\_by (id|price|bedrooms, default id) - sort\_order (asc|desc, default asc)

Success: 200

```
[
 {
 "id": 1,
 "region_id": 1,
 "price": 900.0,
 "bedrooms": 2,
 "listing_type": "rent"
 }
]
```

```
}
]
```

**GET /listings/{listing\_id}** Fetch one listing.

Success: 200

Errors: 404

**PUT /listings/{listing\_id} (auth required)** Update full listing record.

Body:

```
{
 "region_id": 1,
 "price": 1000,
 "bedrooms": 3,
 "listing_type": "rent"
}
```

Success: 200

Errors: 401, 403, 404, 409, 422

**DELETE /listings/{listing\_id} (auth required)** Delete listing.

Success: 204

Errors: 401, 403, 404

### 6.3 Analytics

**GET /analytics/** Root analytics message.

Success: 200

```
{
 "message": "Analytics endpoints"
}
```

**GET /analytics/regions/{region\_id}/median-rent** Median monthly rent (rent listings only).

Success: 200

```
{
 "region_id": 1,
 "median_rent": 900.0
}
```

Errors: 404 (no rent data)

**GET /analytics/regions/{region\_id}/affordability** Affordability for one region.

Success: 200

```
{
 "region": "Leeds",
 "median_monthly_rent": 900.0,
 "median_annual_rent": 10800.0,
 "average_income": 30000.0,
 "affordability_index": 0.36,
 "classification": "Moderate stress"
}
```

Errors: 404

**GET /analytics/regions/{region\_id}/affordability/simulate** What-if affordability simulation.

Query params: - monthly\_rent (optional, >0) - average\_income (optional, >0) - rent\_change\_pct (default 0, range -100..1000) - income\_change\_pct (default 0, range -100..1000)

Success: 200

```
{
 "region": "Leeds",
 "baseline": {
 "median_monthly_rent": 900.0,
 "average_income": 30000.0,
 "affordability_index": 0.36,
 "classification": "Moderate stress"
 },
 "scenario": {
 "monthly_rent": 990.0,
 "average_income": 28500.0,
 "rent_change_pct": 10.0,
 "income_change_pct": -5.0,
 "affordability_index": 0.42,
 "classification": "Moderate stress"
 },
 "delta": {
 "index_change": 0.06,
 "classification_changed": false
 }
}
```

Errors: 404, 422

**GET /analytics/regions/{region\_id}/price-trend** Price trend summary by bedroom count.

Query params: - listing\_type (optional, rent|sale)

Success: 200

```
{
 "region": "Leeds",
 "listing_type_filter": "rent",
 "total_listings": 2,
 "overall_avg_price": 900.0,
 "overall_min_price": 800.0,
 "overall_max_price": 1000.0,
 "by_bedrooms": [
 {
 "bedrooms": 2,
 "listing_count": 2,
 "avg_price": 900.0,
 "min_price": 800.0,
 "max_price": 1000.0
 }
]
}
```

Errors: 404, 422

**GET /analytics/regions/{region\_id}/risk-score** Composite housing risk score for a region using rent burden, rent volatility, and price level.

Success: 200

```
{
 "region": "Leeds",
 "risk_score": 58.2,
 "risk_band": "Medium",
 "affordability_index": 0.36,
 "median_monthly_rent": 900.0,
 "sample_size": 5,
 "factors": {
 "affordability_component": 36.0,
 "volatility_component": 10.2,
 "price_level_component": 12.0,
 "coefficient_of_variation": 0.204
 }
}
```

Errors: 404 (insufficient data)



**GET /analytics/affordability/rankings** Affordability results for all regions sorted ascending by affordability index.

Success: 200 (array of affordability result objects)

Errors: 404 when no affordability data exists

## 7. Notes

- `listing_type` supports only: `rent`, `sale`.
- Schema validation rejects invalid inputs with standardized 422 response.
- Delete endpoints return 204 with empty body on success.

## 8. Supporting Scripts

These scripts support reproducible data and performance workflows alongside the API:

- Prepare external dataset into normalized internal CSVs:

```
python3 scripts/prepare_external_dataset.py \
--source-csv path/to/external_housing_data.csv \
--out-regions data/prepared_regions.csv \
--out-listings data/prepared_listings.csv
```

- Import normalized CSVs and generate ingestion provenance manifest:

```
python3 scripts/import_data.py \
--regions data/prepared_regions.csv \
--listings data/prepared_listings.csv \
--manifest data/ingestion_manifest.json
```

- Benchmark key read endpoints:

```
python3 scripts/benchmark_api.py --iterations 20
```